

Doc-Doctor 项目实施报告

1. 项目概述

1.1 项目信息

- **项目名称：**Doc-Doctor
- **项目类型：**Visual Studio Code 扩展插件
- **开发语言：**TypeScript、C++、JavaScript
- **项目状态：**已完成核心功能实现

1.2 实现目标

本项目实现了以下核心功能：

1. C/C++ 项目注释完整性检查
2. 问题列表展示与导航
3. Git 函数内容变更警告
4. AI 辅助更新注释
5. 白名单配置管理
6. 数据持久化存储

2. 技术实现

2.1 技术架构实现

系统采用前后端分离的分层架构，各层实现如下：

2.1.1 前端层实现

技术栈：HTML/CSS/JavaScript + VS Code Webview UI Toolkit

核心文件：

- `media/sidebar.js`：前端交互逻辑，处理用户操作和消息通信
- `media/toolkit.js`：VS Code Webview UI Toolkit 组件库

实现要点：

- 使用 `vscode.postMessage()` 与扩展通信
- 通过 `window.addEventListener('message')` 接收扩展消息
- 使用 Webview UI Toolkit 组件构建界面
- 实现问题列表的筛选、搜索、状态管理功能

2.1.2 扩展主体层实现

技术栈： TypeScript + Node.js

核心文件：

- `src/extension.ts`：扩展入口，注册命令和视图
- `src/modules/projectCheck.ts`：项目检查核心逻辑
- `src/modules/fileCheck.ts`：文件解析模块
- `src/modules/functionCheck.ts`：函数检查模块
- `src/modules/jumpToLocation.ts`：代码跳转模块
- `src/modules/aiRefactor.ts`：AI 辅助模块
- `src/modules/filewhiteList.ts`：白名单过滤模块
- `src/modules/database.ts`：数据库桥接层

实现要点：

- 使用 VS Code Extension API 访问工作区文件
- 通过 `postMessage` 机制与 Webview 通信
- 实现异步文件扫描和解析
- 支持取消检查和进度提示

2.1.3 数据层实现

技术栈： TypeScript + C++ + SQLite3

核心文件：

- `src/modules/database.ts`：TypeScript 桥接层
- `native/src/database.cpp`：C++ 数据库实现
- `native/src/database.h`：C++ 头文件

实现要点：

- 使用 `koffi` 库实现 FFI 调用
- C++ 层封装 SQLite3 操作
- 通过 JSON 格式进行数据序列化
- 数据库文件存储在 `.doc-doctor/problems.db`

2.2 核心功能实现

2.2.1 函数解析实现

实现方式： 使用正则表达式匹配 C/C++ 函数定义

核心代码逻辑：

```
// 函数定义正则表达式
const functionRegex = /([\w\s\*]+?)\s+([A-Za-z_][A-Za-z0-9_]*\s*\([^)]*\)\s*\{}/g;

// 提取函数信息
function extractFunctions(content: string): FunctionInfo[] {
    // 匹配函数定义
    // 向上搜索注释块
    // 计算函数位置（行号、列号）
}
```

实现难点：

- 需要过滤控制语句（if、for、while 等）的误匹配
- 需要准确关联函数与注释块
- 需要处理多行函数签名

解决方案：

- 使用更精确的正则表达式模式
- 验证注释块与函数之间的关联性
- 支持多行函数签名的解析

2.2.2 注释检查实现

实现方式：解析 Doxygen 注释块，检查标签完整性

核心代码逻辑：

```
function checkFunctionComment(func: FunctionInfo): ProblemInfo[] {
    const problems: ProblemInfo[] = [];

    // 检查 @brief
    if (!hasBrief(func.comment)) {
        problems.push({ type: BRIEF_MISSING, ... });
    }

    // 检查 @param
    const missingParams = findMissingParams(func);
    missingParams.forEach(param => {
        problems.push({ type: PARAM_MISSING, ... });
    });

    // 检查 @return
    if (needsReturn(func) && !hasReturn(func.comment)) {
        problems.push({ type: RETURN_MISSING, ... });
    }

    return problems;
}
```

实现要点：

- 支持多种注释格式（`/** */`、`/* */`）

- 准确匹配参数名与 `@param` 标签
- 区分 void 和非 void 函数

2.2.3 Git 集成实现

实现方式：使用 VS Code Git 扩展 API

核心代码逻辑：

```
async function detectGitChanges(): Promise<Map<string, string>> {
  // 检测 Git 仓库
  const gitExtension = vscode.extensions.getExtension('vscode.git');
  const git = gitExtension?.exports.getAPI(1);

  // 获取 HEAD~1 版本
  const previousFunctions = await getPreviousVersionFunctions(git);

  // 比较函数体变更
  return compareFunctionBodies(currentFunctions, previousFunctions);
}
```

实现要点：

- 使用 VS Code Git API 访问仓库信息
- 执行 `git diff HEAD~1` 获取变更文件
- 忽略空白和注释变更，只检测实质性变更
- 处理非 Git 仓库或历史不足的情况

2.2.4 AI 辅助实现

实现方式：HTTP 请求调用外部 AI 服务

核心代码逻辑：

```
async function generateCommentWithAI(problem: ProblemInfo): Promise<string> {
  // 构造请求上下文
  const context = buildAIContext(problem);

  // 调用 AI 服务
  const response = await fetch(aiEndpoint, {
    method: 'POST',
    headers: { 'Authorization': `Bearer ${apiKey}` },
    body: JSON.stringify(context)
  });

  // 解析和格式修正
  const comment = await response.json();
  return formatComment(comment);
}
```

实现要点：

- 支持 OpenAI 兼容的 API 格式
- 实现注释格式自动修正

- 提供预览确认机制
- 使用 SecretStorage 加密存储 API Key

2.2.5 数据库实现

实现方式：TypeScript 桥接层 + C++ DLL + SQLite3

核心代码逻辑：

TypeScript 层：

```
function saveProblemToDB(problem: ProblemInfo): Promise<number> {
  const json = JSON.stringify(problem);
  return new Promise((resolve, reject) => {
    const result = nativeLib.saveProblem(json);
    if (result.success) {
      resolve(result.id);
    } else {
      reject(new Error(result.error));
    }
  });
}
```

C++ 层：

```
extern "C" int saveProblem(const char* jsonInput) {
  // 解析 JSON
  auto problem = nlohmann::json::parse(jsonInput);

  // 插入数据库
  sqlite3_stmt* stmt;
  sqlite3_prepare_v2(db, INSERT_SQL, -1, &stmt, nullptr);
  // ... 绑定参数
  sqlite3_step(stmt);

  return sqlite3_last_insert_rowid(db);
}
```

实现要点：

- 使用 koffi 定义 C++ 函数接口
- JSON 序列化/反序列化
- 错误处理和内存管理
- 支持批量插入优化

2.3 性能优化实现

2.3.1 文件扫描优化

- 使用 `workspace.findFiles()` 异步查找文件
- 限制最大文件数量（10,000 个）
- 跳过超过 1MB 的大文件

- 支持取消检查机制

2.3.2 数据库优化

- 批量插入问题记录
- 使用事务提高写入性能
- 限制单次检查最多返回 1,000 条问题

2.3.3 前端优化

- 问题列表分页显示（最多 200 条）
- 使用虚拟滚动（如需要）
- 延迟加载和按需渲染

3. 遇到的问题与解决方案

3.1 技术问题

问题 1：函数解析准确性

问题描述：正则表达式无法准确解析所有 C/C++ 函数定义，特别是模板函数和宏定义函数。

解决方案：

- 优化正则表达式模式，提高匹配准确性
- 增加边界情况处理
- 记录无法解析的函数，在日志中提示
- 后续可考虑引入 Tree-sitter 语法解析器

问题 2：跨语言调用稳定性

问题描述：FFI 调用 C++ DLL 时可能出现内存问题或崩溃。

解决方案：

- 完善的错误处理和异常捕获
- 使用 JSON 进行数据序列化，避免直接传递指针
- 充分的单元测试和集成测试
- 提供纯 TypeScript 备选方案 (better-sqlite3)

问题 3：Git API 可用性

问题描述：非 Git 仓库或历史不足时，Git 功能不可用。

解决方案：

- 实现优雅降级，检测 Git 可用性
- 提供配置开关，允许用户禁用 Git 功能
- 非 Git 场景下不影响基础检查功能

问题 4: AI 服务响应时间

问题描述: AI 服务响应可能较慢, 影响用户体验。

解决方案:

- 实现超时机制 (默认 60 秒)
- 显示加载状态和进度提示
- 支持取消请求
- 缓存常见函数的注释生成结果 (可选)

3.2 架构问题

问题 1: 前后端通信复杂度

问题描述: Webview 与 Extension 之间的消息通信需要手动管理消息类型。

解决方案:

- 定义统一的消息类型枚举
- 实现消息路由机制
- 使用 TypeScript 类型检查确保消息格式正确

问题 2: 配置管理

问题描述: 配置项分散在多个地方, 难以统一管理。

解决方案:

- 集中定义配置项接口
- 使用 VS Code 配置 API 统一管理
- 实现配置变更监听和自动同步

3.3 用户体验问题

问题 1: 大项目检查耗时

问题描述: 大项目检查时间较长, 用户等待体验差。

解决方案:

- 实现进度提示, 显示当前检查的文件
- 支持取消检查操作
- 优化文件扫描和解析性能
- 考虑增量检查机制

问题 2: 问题列表展示

问题描述: 问题数量多时, 列表渲染性能差。

解决方案:

- 限制前端显示数量 (最多 200 条)
- 实现分页或虚拟滚动

- 优化 DOM 操作和事件处理

4. 测试实现

4.1 单元测试

测试框架： Mocha + Chai

测试覆盖：

- 文件解析模块测试
- 函数检查模块测试
- 白名单过滤测试
- 跳转定位测试

测试文件：

- `src/test/unit/fileCheck.test.ts`
- `src/test/unit/functionCheck.test.ts`
- `src/test/unit/filewhiteList.test.ts`
- `src/test/unit/jumpToLocation.test.ts`

4.2 集成测试

测试内容：

- 数据库操作测试
- 项目检查流程测试
- Git 集成测试

测试文件：

- `src/test/integration/database.test.ts`
- `src/test/integration/projectCheck.test.ts`

4.3 功能测试

测试场景：

- 单文件检查
- 项目全量检查
- 问题筛选和搜索
- 代码跳转
- AI 修复注释
- 设置保存

4.4 性能测试

测试指标：

- 100 个文件检查时间：< 30 秒
- 500 个文件检查时间：< 120 秒
- 数据库插入性能：100 条记录 < 2 秒
- 前端渲染性能：200 条问题 < 1 秒

5. 部署与发布

5.1 构建流程

5.1.1 TypeScript 编译

```
cd doc-doctor
npm install
npm run compile
```

5.1.2 C++ 动态库构建

```
cd doc-doctor/native
cmake -B build -S . --preset=default
cmake --build build --config Release
```

5.1.3 扩展打包

```
npm install -g @vscode/vsce
vsce package
```

5.2 发布流程

1. **版本号管理**：遵循语义化版本（Semantic Versioning）
2. **变更日志**：更新 CHANGELOG.md
3. **打包扩展**：生成 `.vsix` 文件
4. **发布到市场**：上传到 VS Code Marketplace

5.3 运行时文件

扩展运行时会创建以下文件：

```
<workspace>/
├── .doc-doctor/
│   └── problems.db    # SQLite 数据库文件
```

6. 性能指标

6.1 检查性能

项目规模	文件数	检查时间 (P95)	内存占用
小规模	< 100	< 30 秒	< 100 MB
中等规模	100-500	30-120 秒	< 200 MB
大规模	500-1000	120-300 秒	< 500 MB

6.2 数据库性能

操作	数据量	耗时 (P95)
单次插入	1 条	< 50 ms
批量插入	100 条	< 2 秒
查询所有	1000 条	< 500 ms

6.3 前端性能

操作	数据量	耗时
渲染问题列表	200 条	< 1 秒
筛选/搜索	1000 条	< 100 ms
跳转定位	-	< 200 ms

7. 代码质量

7.1 代码规范

- 使用 TypeScript 严格模式
- 遵循 ESLint 代码规范
- 使用 Prettier 格式化代码
- 函数和类添加 JSDoc 注释

7.2 代码统计

- **TypeScript 代码行数**: 约 5,000 行
- **C++ 代码行数**: 约 800 行
- **JavaScript 代码行数**: 约 1,500 行
- **测试代码行数**: 约 2,000 行

7.3 代码覆盖率

- **单元测试覆盖率**: > 70%
 - **集成测试覆盖率**: > 60%
 - **整体测试覆盖率**: > 65%
-

8. 项目总结

8.1 实现成果

1. **核心功能完整**: 所有需求文档中定义的核心功能均已实现
2. **技术方案可行**: 验证了前后端分离、FFI 调用、Git 集成等技术方案
3. **性能满足需求**: 在中等规模项目上性能表现良好
4. **用户体验良好**: 界面简洁, 操作流畅

8.2 技术亮点

1. **跨语言调用**: 成功实现 TypeScript 与 C++ 的 FFI 调用
2. **Git 集成**: 实现了基于 Git 历史的函数变更检测
3. **AI 集成**: 实现了外部 AI 服务的集成和注释生成
4. **数据持久化**: 使用 SQLite 实现了高效的数据存储

8.3 后续改进方向

1. **函数解析准确性**: 考虑引入 Tree-sitter 语法解析器
 2. **性能优化**: 实现增量检查和并行处理
 3. **功能扩展**: 支持更多文件类型 (.h, .hpp) 和注释风格
 4. **用户体验**: 增加更多交互提示和帮助文档
-

9. 附录

9.1 依赖清单

Node.js 依赖:

- `@vscode/vscode`: VS Code API
- `koffi`: FFI 库
- `mocha`、`chai`: 测试框架

C++ 依赖:

- SQLite3
- nlohmann/json

9.2 构建环境要求

- Node.js $\geq 16.0.0$
- npm $\geq 8.0.0$
- CMake ≥ 3.20
- C++ 编译器 (MSVC / GCC / Clang)

9.3 参考文档

- [VS Code Extension API](#)
- [Doxygen 文档](#)
- [SQLite 文档](#)
- [koffi 文档](#)